

# Project Plan – Game Theory Simulation

## 1 General Information

**Title:** Game Theory Simulation Program

**Author:** Adi Dasgupta

**Student Number:** 103552167

**Study Program:** Exchange Studies

**Date:** February 19, 2026

## 2 General Description and Difficulty Level

The project aims to create an interactive graphical simulation based on game theory. The simulation represents a duopoly market where two companies produce the same product and at the same time decide on the quantity of production at each round. The linear demand curve that determines market price is one that is dependent on the aggregate output of the two firms. Every company aims to maximize its cumulative profit in several rounds of the game. The rounds are stochastic in nature; each round is followed by a probability that the game ends. This uncertainty in nature brings in a strategic aspect that forces players to consider both short and long-term implications in their strategies.

The project will be implemented on the most challenging level. In addition to meeting all the baseline requirements, the implementation will include stochastic components, a graphical interface, an automated opponent with a strategic decision rule, and a system for showing the optimal equilibrium values after the simulation is completed. The unique aspect of this project is that it is not only a simulation of the gameplay but also provides analytical commentary by comparing the decisions made by the participant with theoretically optimal solutions based on the existing economic theory.

## 3 Use Case Description and User Interface

The program connects to us via a GUI that is developed in PyQt. The primary window is divided into three primary sections: a graph displaying the demand curve, a control panel where you enter the quantity of production, and a history table where all the past rounds are recorded. In essence, you enter the amount you desire to produce, press submit, and the application then determines the next move of the opponent, calculates the market price, calculates the profit of both firms, and updates all this on the screen.

And thus the first thing you see when you open the app is the main window. All you have to do is to enter your number of production in the field and submit. Inside the program, the program will sum up all the production and take the price of the demand curve and compute the profit of each firm, and dump the figures in the history table. You

are able to see the numbers both in writing and on the graph and the cycle continues until the game decides randomly that it is finished. Figure illustrates the interface layout. 1.

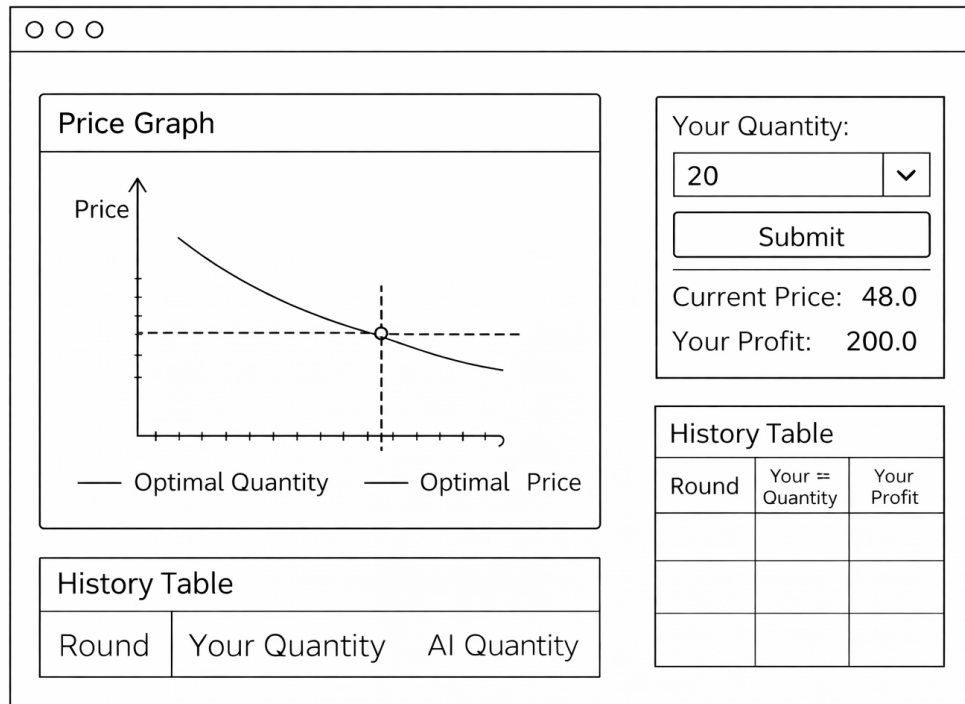


Figure 1: Draft of the graphical user interface

## 4 Program Structure

The program is divided into several logical components that separate user interface functionality, game control logic, economic calculations, and artificial intelligence behavior. The graphical interface is handled by a main window class that manages all user interactions and visual updates. A controller class manages the progression of rounds and coordinates communication between other components. Separate classes represent the market environment, the firms participating in the game, and the module responsible for computing results such as profits and equilibrium values. The artificial intelligence opponent is implemented as a separate strategy class that determines production quantities based on the current state of the game.

This modular design was chosen because it improves maintainability and extensibility. By separating responsibilities into distinct classes, individual components can be modified or tested independently without affecting the rest of the system. The overall class structure of the program is presented in Figure 2.

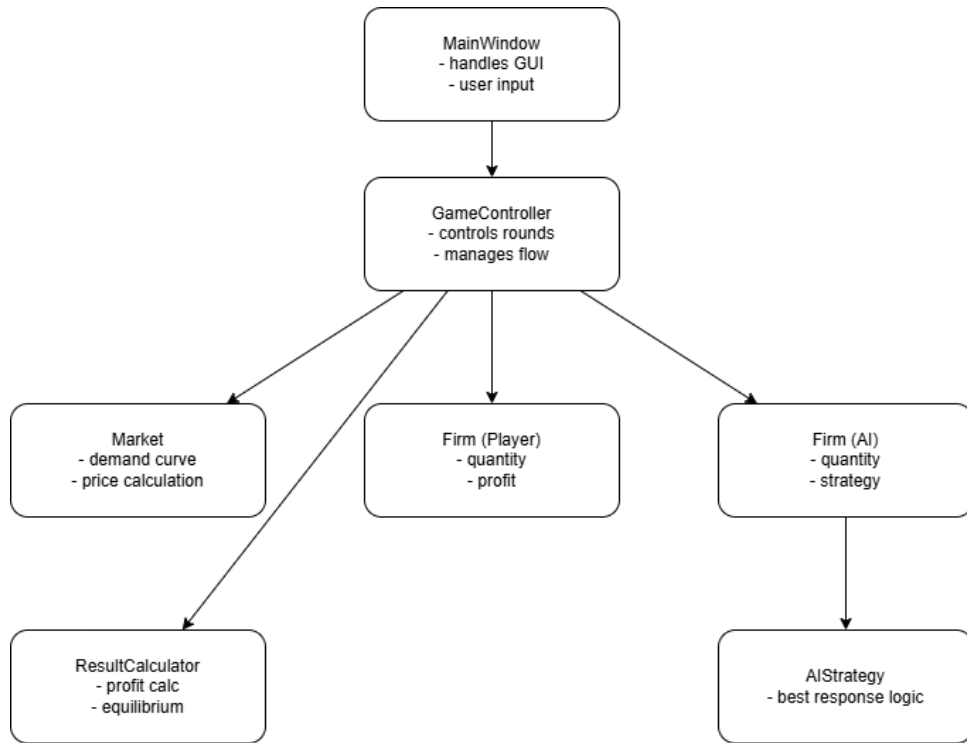


Figure 2: Class diagram of the program

## 5 Data Structures

The data required for the simulation consists mainly of numerical values representing quantities, prices, costs, and profits, as well as records of previous rounds. These values can be efficiently stored using standard Python data structures such as lists, dictionaries, and numeric variables. Lists are suitable for storing the history of rounds because the number of rounds is not known in advance and may change dynamically during execution. Dictionaries are useful for storing economic parameters such as demand coefficients and cost values because they allow values to be accessed using descriptive keys. More complex data structures are not necessary for this project because the problem domain is relatively structured and does not require advanced storage mechanisms.

## 6 Files and File Formats

The program may read configuration data from external text files in order to allow easy modification of parameters without altering the source code. Such files can contain numerical values representing the demand curve parameters and cost coefficients. Text format is chosen because it is simple, human-readable, and easy to parse. The program may also optionally save the results of a simulation to a text file so that users can review their performance later. These files will store round-by-round results such as chosen quantities, resulting prices, and profits.

## 7 Algorithms

The main algorithms used in the program relate to price determination, profit calculation, opponent strategy, and game termination. The market price is calculated using a linear demand function in which the price decreases as total production increases. Profits are calculated by multiplying the difference between price and marginal cost by the firm's chosen quantity. The artificial intelligence opponent determines its quantity using a best-response rule derived from economic theory, meaning it selects the production level that maximizes its profit given the opponent's previous action. The game termination mechanism is based on probability, so after each round, a random number is generated and compared to a predefined threshold to determine whether the simulation ends. At the conclusion of the game, the program calculates the Nash equilibrium quantities analytically and displays them for comparison with the player's choices.

## 8 Testing Plan

Testing will be conducted both during development and after the program is completed. During implementation, individual components such as the price calculation and profit functions will be tested separately using unit tests to ensure they produce correct results for known inputs. After the core functionality has been verified, system-level testing will be performed by running the complete program and checking that all elements interact correctly. The graphical interface will be tested to confirm that user input is processed correctly and that invalid inputs are handled gracefully. Additional tests will verify that prices decrease when total production increases, that profits are computed accurately, and that the game termination mechanism behaves according to its probability setting. These tests ensure that the essential features of the simulation function reliably.

## 9 Libraries and Tools

The project will be implemented in Python using the PyQt library for the graphical user interface. Standard libraries such as random and math will be used for numerical calculations and probability generation. No external libraries beyond those permitted by the course guidelines will be used. In particular, automated interface design tools are avoided in order to ensure that the implementation demonstrates understanding of the underlying concepts.

## 10 Schedule

The implementation will proceed in stages, beginning with the core economic logic, followed by the controller that manages game rounds, and then the graphical interface. Once the basic program is functional, the AI opponent will be implemented and integrated. Testing will be carried out continuously as the components are completed. By the third checkpoint, the fundamental classes and calculation logic will be operational, and the graphical interface will be able to display basic information. By the sixth checkpoint, the simulation will be fully playable, and at least one unit test will be implemented. Final development time will be devoted to debugging, polishing the interface, and refining the user experience.

1. Aalto University external study material on demand, supply, duopoly, and Nash equilibrium:  
[http://aalto-econ.fi/tervio/kandimikro/external\\_materials.html](http://aalto-econ.fi/tervio/kandimikro/external_materials.html)